

Setup & Deployment Guide

Load the label-free network, wire telemetry and operator feedback, and run the advisory scorer in production

The production runbook: how the Entry runner loads the emitted bundle, how to configure telemetry inputs and the feedback channel, how advisories leave the system, how continuous learning runs without a redeploy, and the safety gates you must keep in place.

Document	Setup & Deployment Guide
Product	Jneopallium Self-Supervised Maintenance Guardian
Entry point	com.rakovpublic.jneuropallium.worker.application.Entry
Bundle	model/self-supervised-maintenance
Modes	local · http · grpc
Safety	ADVISORY / read-only
Author	Dmytro Rakovskyi — Kharkiv, Ukraine
Date	2 July 2026
License	BSD 3-Clause

Contents

1. What you are deploying
2. Prerequisites
3. Step 1 — the bundle and the IContext
4. Step 2 — run the Entry point
5. Step 3 — wire telemetry inputs
6. Step 4 — wire the feedback channel
7. Step 5 — where advisories go
8. Step 6 — deployment modes
9. Step 7 — continuous learning in production
10. Rollout: shadow → advisory
11. Operations and monitoring
12. Safety checklist
13. Troubleshooting

1 What you are deploying

A running Jneopallium worker that ingests asset telemetry, scores each frame through the label-free maintenance network, and emits read-only advisories with a fault family and a lead time. It also consumes operator feedback and adapts its own alert thresholds in place — no restart, no redeploy. It never actuates a device.

The one rule that never changes

This model is advisory. It recommends inspection; it does not schedule work, trip an interlock, or move an actuator. The hard safety layer stays entirely outside the learned path.

2 Prerequisites

- JDK 17 and the built worker artifact (worker-1.0-SNAPSHOT.jar) or your packaged model JAR.
- The trained bundle from the Training Guide under model/self-supervised-maintenance/.
- A telemetry source (OPC UA or MQTT) and, for learning, a feedback source (the UI/CMMS event the operator marks).
- Writable storage for persisted neuron state (so learning survives restarts).

3 Step 1 — the bundle and the IContext

The deployable model is the bundle directory. production-context.json is the IContext the runner deserializes; its key entries:

IContext key	Value / meaning
configuration.input.layermeta	the five layers (0–4)
neuronnet.classes	the four ssmaint neuron classes (validated at startup)
input.inputs	AssetTelemetrySignal source (OPC UA / MQTT)
outputAggregator	where advisories are written (JSONL / Kafka / MQTT)
storage.json	model/self-supervised-maintenance (persisted state)
processing.frequency.map / slowfast.ratio	fast telemetry vs slow feedback cadence
isteacherstudying=false, discriminatorsAmount=0, infiniteRun=true	advisory inference loop
safetyMode / advisoryOnly	ADVISORY / true — enforced

The runner validates neuronnet.classes at startup and fails fast if a class is missing — so a misconfigured bundle never half-starts.

4 Step 2 — run the Entry point

```
java -cp worker-1.0-SNAPSHOT.jar \
  com.rakovpublic.jneuropallium.worker.application.Entry \
  local <model-jar-url> <IContext-class> \
  worker/src/main/resources/model/self-supervised-maintenance/production-context.json
```

The Runner deserializes the IContext, loads your model JAR via the JAR class loader, selects the IApplication for the chosen mode, and begins the inference loop. From here it is telemetry in, advisories out.

5 Step 3 — wire telemetry inputs

Map each asset's live tags to an AssetTelemetrySignal per tick: the eight sensor channels plus the operating-regime integer. Use the standard industrial inputs:

- **OPC UA** — OpcUaMeasurementInput for process tags (pressure, flow, temperature, power, valve position).
- **MQTT / Sparkplug B** — MqttMetricInput for IIoT gateways and vibration/acoustic feeds.
- Derive the **regime** from a load or power band if you do not publish it directly.

Get the sensor names right

The reconstruction weights are keyed by sensor name. The tag→sensor mapping must match the sensorOrder in layer-1 exactly, or the residuals are meaningless. Validate the mapping once at commissioning.

6 Step 4 — wire the feedback channel

This is what makes the model improve. When an operator dispositions an advisory in your UI, CMMS, or SIEM, publish an OperatorFeedbackSignal carrying: the asset, the fault family, confirmed / false-positive, the operator identity, and the domain-shift value observed when the advisory fired (so the adapter can freeze during novelty). Feedback runs on the slow loop; it need not be real-time.

- **Confirmed** advisories slightly relax that family's threshold.
- **False-positive** advisories raise it — bounded and rate-limited.
- Only **authenticated** operator feedback should be admitted; tag provenance for audit.

7 Step 5 — where advisories go

Advisories leave through an IOutputAggregator, instantiated by the runtime. Choose per site:

Aggregator	Use it for
JsonlResultAggregator	file/SIEM ingest, simplest first deployment
KafkaAdvisoryOutputAggregator	streaming into an enterprise bus / historian
MqttAdvisoryOutputAggregator	publishing back to an IIoT broker / dashboard

Each MaintenanceAdvisorySignal carries the asset, fault family, severity, estimated lead time, uncertainty, and a human-readable recommendation. advisoryOnly is always true on the wire.

8 Step 6 — deployment modes

Mode	Shape	When
------	-------	------

local	single worker in one JVM	a plant, a cell, an edge box
http	master + HTTP workers	several lines, central coordination
grpc (client/master)	master + gRPC workers	larger fleets, lower-latency fan-out

The model is identical across modes; only the transport and worker topology change. Start local, scale out when the asset count warrants it.

9 Step 7 — continuous learning in production

Learning at runtime lives in neuron state and is persisted to storage.json, so it survives restarts without a rebuild. The recommended discipline for evolving anything heavier than thresholds is champion/challenger:

- the **champion** (current, stable) serves advisories;
- a **challenger** adapts on the harvested feedback in shadow;
- promote the challenger only when it beats the champion on **independent** outcomes (actual work orders / confirmed failures), and keep the champion snapshot for instant rollback.

No redeploy required

Threshold adaptation is applied in place via ThresholdUpdateSignal into the running gate. Promotion of a challenger is an internal, audited state swap — not a JAR redeploy.

10 Rollout: shadow → advisory

- **Shadow (weeks 1–n)**. Advisories are recorded but not shown to operators; you learn the false-positive rate and let baselines settle. Feedback can still be harvested from actual outcomes.
- **Advisory (go-live)**. Operators see advisories and disposition them; the feedback loop starts driving nuisance alerts down.
- There is no autonomous stage. The model never graduates out of advisory.

11 Operations and monitoring

- Track the **false-positive rate per family** and confirm it trends down as feedback accumulates.
- Watch **domain-shift / uncertainty**; a rising trend means the plant has moved away from the training window — schedule a refit (Training Guide).
- Snapshot persisted neuron state on a schedule so a promotion can be rolled back.
- Alert on **detection lead time** shrinking — advisories firing too close to failure suggest the trend or thresholds need attention.

12 Safety checklist

- advisoryOnly is true and cannot be disabled (SsMaintConfig invariant).
- No output path reaches an actuator; interlocks and the hard safety gate are outside this model.
- Feedback is authenticated and provenance-tagged; adaptation freezes during domain shift.
- Persisted state is snapshotted; challenger promotions are audited and reversible.

13 Troubleshooting

Symptom	Likely cause	Fix
Startup fails fast	neuronnet.classes / bundle mismatch	Re-emit bundle; check classpath
Residuals nonsensical	tag→sensor mapping wrong	Align mapping with layer-1 sensorOrder
Thresholds never move	feedback not arriving / frozen	Check feedback wiring & domain-shift
Learning lost on restart	storage not persisted	Set writable storage.json

Deploy it in shadow, wire the feedback, watch the false-positive rate fall — and never let it leave advisory. The safety layer is deterministic and lives outside the model, exactly as it should.